How Data-Structure Choice Affects the Performance of a Limit-Order Matching Engine

Oct 2, 2026 · @yash

Abstract

The same matching engine ran up to 3.9 times faster depending only on its data structures, even though every version had the same Big-O complexity. This report asks: *how does data-structure choice affect the latency and throughput of a limit-order matching engine?*

I built a price-time priority matching engine in C++17 and four interchangeable versions of it (A to D). They share one interface and one set of matching rules, and differ only in how they store price levels, order queues and the order index. All four passed the same 40 tests, and a differential test confirmed they produce identical trades. I benchmarked each on 1,000,000 actions at book sizes from 1,000 to 1,000,000 resting orders, on a standard order flow and a harder one, with five interleaved runs per measurement.

Two changes caused most of the gain: an open-addressing hash map for the order index (+53% to +79% throughput) and a flat array for price levels (+19% to +86%). Engine D, which combines both, was 2.8 to 3.9 times faster than the standard-library baseline across both flows, and its throughput never changed by more than 13% between them. Engine B was fastest on the easy flow but lost 44% to 73% of its throughput on the hard flow. The answer is that data structures matter through constant factors and memory layout rather than Big-O, and that the best structure depends on the workload it is tested on.

1. Introduction

Every electronic exchange runs a matching engine: the program that receives buy and sell orders and decides which of them trade, at what price and in what order. It keeps an order book of all orders waiting to be filled. When a new order arrives, the engine checks whether it can trade against the best waiting orders on the other side, records any trades, and stores whatever is left.

Speed matters for two reasons. Throughput, the number of orders processed per second, decides whether the engine keeps up during busy periods. Latency, the time taken to handle one order, decides how quickly traders see the result of their actions. Tail latency, the slowest 1% or 0.1% of orders, matters too, because one slow order can delay every order queued behind it.

Computer science usually compares data structures by their Big-O complexity, which describes how cost grows as the input grows. However, two structures with the same Big-O can perform very differently in practice, because of memory allocation and how well they use the CPU cache. This project tests how much that matters in a realistic program.

Research question: *How does data-structure choice affect the latency and throughput of a limit-order matching engine?*

To answer it, I built one engine, made four versions that differ only in their data structures, checked that they behave identically, and measured them under controlled conditions.

2. Background

The order book. An order book has two sides. Bids are orders to buy, and the best bid is the highest price. Asks are orders to sell, and the best ask is the lowest price. A *limit order* names a price and waits in the book if it cannot trade straight away. A *market order* names no price and trades at the best available prices until it is filled or the book runs out.

Price-time priority. When several resting orders could fill an incoming order, the engine chooses by two rules applied in order. First, the best price trades first. Second, among orders at the same price, the oldest trades first. Each price level therefore behaves like a queue.

What the engine must do quickly. Every action needs one or more of these operations:

Operation	Needed for	What the engine must find
Find the best price level	Every incoming order	The highest bid or lowest ask
Add to the back of a queue	New limit orders that rest	The queue at one price
Remove from the middle of a queue	Cancels and modifies	One order by its ID

Operation	Needed for	What the engine must find
Look up an order by ID	Cancels and modifies	Where that order is stored

Data structures and Big-O. A sorted tree such as `std::map` finds the best price in O(1) and inserts a new level in O(log L), where L is the number of price levels. A flat array indexed by price does both in O(1), but uses memory for every possible price in its range. A linked list removes an order from the middle in O(1) if you already know where it is. A hash map finds an order by ID in O(1) on average. In my tests the book had only about 300 to 400 price levels, so log L is roughly 9, and every engine does each operation in O(1) or O(log L).

Why Big-O is not the whole story. Modern CPUs read memory through small, fast caches. Data that sits close together in memory, or was used recently, is fast to read. Data scattered across memory, such as linked-list nodes or tree nodes each allocated separately, often has to be fetched from slower main memory, which can take around 100 times longer than a cache hit. Allocating memory for each new order also takes time. These are constant factors that Big-O ignores, and they are what this project measures.

3. Engine design

The engine supports limit and market orders, cancellation, modification, partial fills and a timestamped trade history. It also reports the best bid, best ask, spread and market depth, and rejects duplicate order IDs and zero-quantity orders. It is written in C++17, with Python used only to analyse results and draw charts.

Three design decisions apply to every version:

1. **Prices are stored as whole numbers of cents.** $101.20 is stored as 10120. Floating-point numbers cannot represent many decimal values exactly, and an exchange cannot afford to be wrong by a fraction of a cent.
2. **Trades happen at the resting order's price.** If a buyer bids $101.50 and the best ask is $101.20, the trade happens at $101.20, the price that was already in the book.
3. **Leftover quantity depends on order type.** Unfilled limit-order quantity rests in the book; unfilled market-order quantity is discarded, since there is nothing left to trade against.

Baseline profiling. Before comparing data structures, I benchmarked the baseline engine alone. Three findings shaped the experiment:

- Throughput fell 2.8 times between a 1,000-order book and a 1,000,000-order book, even though the number of price levels stayed at about 300 to 400. Big-O therefore could not explain the slowdown; memory and cache behaviour were the likely cause.
- Each resting order took about 130 bytes, although the order record itself is only 40 bytes. The rest was list pointers, the index entry and memory-allocator overhead.
- The worst-case latency of about 1 ms came from the trade-history vector doubling its capacity and copying itself. I fixed this by reserving space in advance, in every engine, so that the comparison measures the data structures rather than this one bug.

These findings suggested three targets for improvement: memory allocation, how the order index is stored, and how scattered the data is in memory.

4. Method

4.1 Engines. To isolate the effect of data structures, I built four engines that share the same interface and the same matching rules. They differ only in how they store price levels, the orders queued at each level, and the index used to find an order by its ID.

Engine	Price levels	Order queue	Order index
A (baseline)	`std::map` (sorted tree)	`std::list`	`std::unordered_map`
B	Flat array indexed by price tick	Vector, with cancelled slots removed in batches	Array indexed by order ID
C	`std::map`	Intrusive linked list, orders from a memory pool	Open-addressing hash map
D	Flat array indexed by price tick	Intrusive linked list, orders from a memory pool	Open-addressing hash map

C was built from A one optimisation at a time: a memory pool, then an intrusive list, then an open-addressing hash map, then a more compact memory layout. I benchmarked after each step, an approach called an *ablation*, so the effect of each change could be measured separately. D is C with B's flat price array, which isolates the effect of the price-level structure.

4.2 Correctness. Before measuring speed, I checked that every engine behaves identically. Each engine passes the same 40 unit tests. A *differential test* feeds the same seeded order flow to every engine and checks that they produce identical trades and identical final books. Every benchmark run also recorded a checksum of its trades, and all 220 runs matched. Any speed difference therefore reflects the data structures, not different behaviour.

4.3 Order flows. Each engine was tested on two synthetic order flows:

- The **standard flow** contains about 48% limit orders, 5% market orders, 37% cancellations and 10% modifications, spread over about 300 to 400 price levels, with sequential order IDs.
- The **hard flow** uses random 64-bit order IDs and up to 10,000 price levels, with orders clustered within 20 cents of the mid price and 5% placed far away.

Both flows are generated from a fixed random seed, so the same seed always produces the same sequence. Before each run the book was pre-filled to 1,000, 10,000, 100,000 or 1,000,000 resting orders.

4.4 Benchmark procedure. Each run processed 1,000,000 actions after two warm-up runs. Every configuration was measured five times, and the engines' runs were interleaved so that background activity on the computer affected all engines equally. I report the median and the range from the slowest to the fastest run. A difference is treated as real only if the two ranges do not overlap; otherwise it is labelled *noise*.

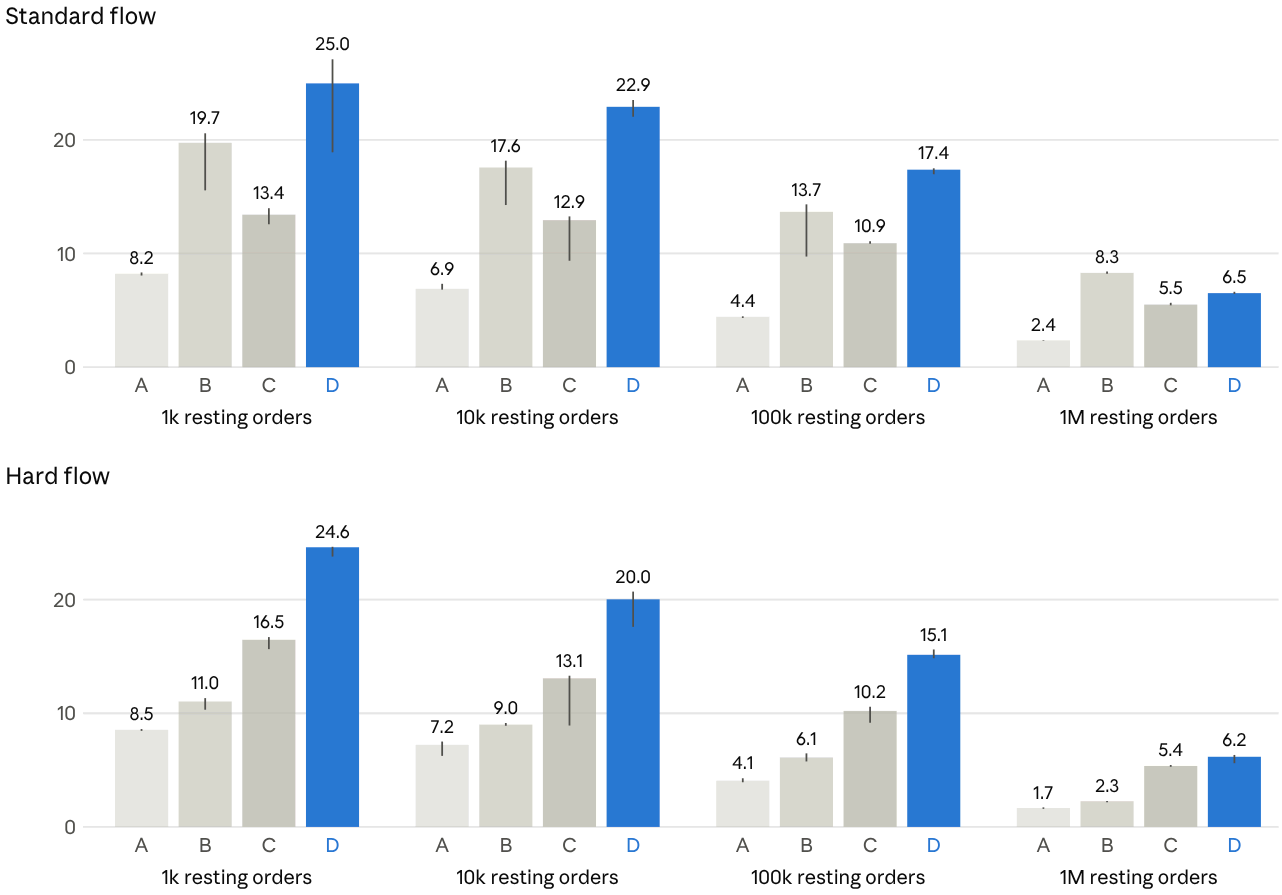
4.5 Metrics and setup. I measured throughput in millions of actions per second; latency at the 50th, 99th and 99.9th percentiles and the worst case (the 99th percentile, or p99, is the latency that 99% of actions were faster than); and book memory, measured as process memory growth using the Windows API. Each action was timed with the CPU's cycle counter, because the Windows clock only ticks every 100 ns; this adds about 6 ns to each measurement. All tests ran single-threaded at high priority on one core, on an Intel i7-14650HX laptop with Windows 11 and GCC 16.2 at `-O3`.

5. Results

Engine D had the highest throughput on the hard flow at every book size, and on the standard flow at every size except 1,000,000, where B was 27% faster.

Only engine D stays fast on both flows; B's lead disappears on the hard flow

Throughput in million actions per second. Bar = median of 5 runs; whisker = slowest to fastest run.



Phase 4 benchmark, second session · 4 engines × 4 book sizes × 2 flows × 5 runs

5.1 Rankings. Using the noise rule from section 4.4, a "~" means the two neighbouring engines' run ranges overlap.

Book size	Standard flow	Hard flow
1k	D ~ B > C > A	D > C > B > A
10k	D > B > C > A	D > C ~ B > A
100k	D > B ~ C > A	D > C > B > A
1M	B > D > C > A	D > C > B > A

5.2 Ablation. Throughput on the standard flow in millions of actions per second, with each step's change measured against the step before it.

Book size	A	+ pool	+ intrusive list	+ hash map	+ layout (= C)	+ flat levels (= D)
1k	8.21	8.15 (noise)	8.92 (noise)	13.68 (+53%)	13.41 (noise)	24.96 (+86%)
10k	6.88	6.53 (−5%)	7.92 (+21%)	12.72 (+61%)	12.93 (noise)	22.90 (+77%)
100k	4.42	4.34 (noise)	6.04 (+39%)	10.82 (+79%)	10.90 (noise)	17.37 (+59%)
1M	2.35	2.71 (+16%)	3.30 (+22%)	5.41 (+64%)	5.50 (noise)	6.51 (+19%)

The hash map was the largest single gain at every size. The flat price array was the second largest, and its gain shrank as the book grew. The memory pool and the compact layout had no reliable effect: in an earlier benchmark session the pool step was within noise at all four sizes.

5.3 Robustness. Change in each engine's throughput when moving from the standard flow to the hard flow.

Engine	1k	10k	100k	1M
A	+4%	noise	−8%	−29%
B	−44%	−49%	−55%	−73%
C	+23%	noise	−6%	noise
D	noise	−13%	−13%	−5%

Every engine changed somewhere, but D never by more than 13%. B lost the most at every size: on the hard flow it was only 1.25 to 1.5 times faster than A, down from 2.4 to 3.5 times on the standard flow.

5.4 Latency. Typical (p50) latency at 1,000,000 resting orders was 403 ns for A, 45 ns for B, 195 ns for C and 135 ns for D on the standard flow. On the hard flow B's rose to 411 ns, close to A's 567 ns, while C and D stayed at 193 ns and 146 ns. D had the best or tied-best p99.9 latency at every size on both flows. B's p99.9 on the hard flow was worse than A's at 1k and 10k. Worst-case latencies overlapped between engines in most cases; the exception was the hard flow at 1M, where B's worst case (0.56 to 0.69 ms) was clearly above C's and D's (under 0.2 ms).

5.5 Memory. A, C and D kept a steady memory footprint: after 1,000,000 further actions with the same number of resting orders, their book memory grew by at most 0.4 MB. B's grew steadily, from 0.27 MB to 15.7 MB at 1k on the standard flow and from 90 MB to 124

MB at 1M. On the hard flow B also needed 1.33 MB for a 1k book, against 0.14 MB for A, because its arrays must cover the wider price range.

6. Discussion

Big-O did not predict the results. All four engines perform each operation in O(1) or O(log L), with L only about 400, yet D was up to 3.9 times faster than A. The differences come from constant factors: how many separate memory allocations an action needs, and how scattered the data it touches is in memory. These causes are inferred from the per-action timings; I did not measure cache misses directly.

The order index mattered most. Replacing `std::unordered_map` with an open-addressing hash map was the largest single gain, +53% to +79%. A standard-library hash map stores each entry in its own separately allocated node, so a lookup typically follows a pointer to a random place in memory. An open-addressing table stores entries directly in one array, so a lookup usually touches only one or two nearby memory locations. The cost is memory: the table is kept at most half full, which raised memory at 1M orders from 86 MB to 116 MB.

Price-level structure mattered second. Swapping C's `std::map` for a flat array (engine D) added +19% to +86%. Finding the level for a price becomes a single array access instead of a walk down a tree. The gain shrank as the book grew, probably because with a million orders, time spent on the orders themselves outweighs time spent finding their level.

My allocation hypothesis was wrong. After profiling the baseline, I expected memory allocation to be the main cost at small book sizes. The ablation did not support this: adding a memory pool had no reliable effect across two benchmark sessions. The intrusive list helped more (+21% to +39% above 1k), but that step also moved orders into a pre-allocated array, so I cannot separate the effect of the list from the effect of storing orders closer together.

Fast is not the same as robust. B was the fastest engine on the standard flow, but its speed depended on conditions that flow happened to provide. Its order index is an array indexed by order ID, which works well when IDs are small and sequential. Its price array is cheap when prices sit in a narrow band. On the hard flow, with random IDs and up to 10,000 price levels, B lost 44% to 73% of its throughput, its p99.9 latency became worse than the baseline's at 1k and 10k, and its memory kept growing while the book stayed the same size. D, which uses a hash map for the index and keeps B's flat price array, changed by at most 13%.

Why B still won at 1M on the standard flow. At 1,000,000 orders B was 27% faster than D. Most of the gap is in new orders: a new limit order took 41 ns in B and 130 ns in D. A likely explanation is that B appends each new order to the end of a vector it has just used, which is already in the cache, while D writes to a random hash-table slot and a separate order node. Again, this is inferred rather than measured.

Taken together, the results suggest that the right question is not "which data structure is fastest?" but "which is fastest for the workload it will actually face?". A benchmark on a single, convenient order flow would have named B the winner.

7. Limitations

The main limitation is that the causes in section 6 are inferred, not measured: without hardware counters for cache misses, I can show which change made an engine faster but only argue why.

- **One machine, one compiler, one operating system.** Different hardware could change the size of the effects or even the ranking.
- **Synthetic order flow.** Both flows follow fixed rules; real markets cluster orders and cancellations in ways these do not capture.
- **The hard flow changes three things at once:** random IDs, a wider price band and clustering. I cannot say which of them hurt B most.
- **Only five runs per measurement.** The overlap rule is a simple, conservative check, not a formal statistical test. Two benchmark sessions disagreed on effects smaller than about 15%, so those should not be relied on.
- **The ablation follows one order.** Adding the optimisations in a different order might credit them differently, and the intrusive-list step also moved orders into a pre-allocated array.
- **The memory pool came from the standard library.** Its lack of effect says nothing about a hand-written pool.
- **Shared overhead in every engine.** A clock read per new order, a function-call layer per action and the timer's 6 ns all shrink the relative gaps between engines.
- **Every engine was told the expected book size in advance** and had its trade history pre-sized. Without these hints, the growth spikes seen in profiling would return.
- **Some worst-case spikes remain unexplained.** After the trade-history fix, A's worst case at 1k still reached 2.9 ms in one run.

- **Correctness was tested, not proven:** 40 unit tests, three seeded flows in the differential test, and matching trade checksums in all 220 runs.

8. Conclusion

Data-structure choice changed the throughput of the same matching engine by up to 3.9 times, even though every version had the same Big-O complexity. The order index and the price-level structure caused most of the difference: an open-addressing hash map and a flat price array together made engine D 2.8 to 3.9 times faster than the standard-library baseline, while the memory pool and compact layout had no reliable effect.

The experiment also showed that a structure's advantage can depend on the workload. Engine B was fastest on the standard flow but lost up to 73% of its throughput on the hard flow, while D changed by at most 13%. For a matching engine, the best data structure is therefore not simply the fastest one in a single benchmark, but the one that stays fast across the order flows it will meet.

9. Future work

1. **Measure cache misses directly** with hardware counters (for example `perf` on Linux), to test the explanations in section 6 instead of inferring them.
2. **Separate the hard flow's three factors** by changing only the IDs, only the price band, or only the clustering at a time, to find which one hurts B.
3. **Try the missing combinations:** B with C's hash map, and D with an ID-indexed array, to see whether D's remaining gap to B at 1M can be closed.
4. **Test on real market data** instead of synthetic flows, and on more than one machine.
5. **Repeat the ablation in a different order** to check that each optimisation's credit does not depend on when it was added.